



| Academic Year | Module                                                   | Assessment Number | Assessment Type |
|---------------|----------------------------------------------------------|-------------------|-----------------|
| 2026          | 6CS012/HJ1: Artificial Intelligence and Machine Learning | 3                 | Report          |

---

## Sentiment Analysis of Book Reviews

---

Using RNN, LSTM, and Word2Vec Embeddings

Student Id : 2408841  
Student Name : Samartha Shakya  
Section : L6CG10  
Module Leader : Mr. Siman Giri  
Tutor : Mr. Anish Khatiwada  
Submitted on : 10-05-2026

## Table of Contents

|                                                    |    |
|----------------------------------------------------|----|
| Introduction.....                                  | 1  |
| Dataset .....                                      | 1  |
| Source and Overview .....                          | 1  |
| Sentiment Label Mapping.....                       | 2  |
| Methodology .....                                  | 2  |
| Text Preprocessing.....                            | 2  |
| Sequence Length Analysis.....                      | 3  |
| Tokenization and Padding.....                      | 3  |
| Splits and Class Weights.....                      | 3  |
| Model Architectures.....                           | 3  |
| Model 1 – Simple RNN with Trainable Embedding..... | 3  |
| Model 2 – BiLSTM with Trainable Embedding.....     | 4  |
| Model 3 – BiLSTM with Word2Vec Embeddings .....    | 4  |
| Experiments and Results.....                       | 5  |
| Dataset Visualization .....                        | 5  |
| Word Cloud Visualization .....                     | 5  |
| Review Length Distribution.....                    | 6  |
| Overall Model Performance.....                     | 6  |
| Training Curves .....                              | 7  |
| Per-Class Performance.....                         | 8  |
| Model 1 – Simple RNN .....                         | 8  |
| Model 2 – BiLSTM Trainable Embedding.....          | 8  |
| Model 3 – BiLSTM + Word2Vec .....                  | 8  |
| Embedding Comparison.....                          | 11 |
| Computational Efficiency .....                     | 11 |
| Error Analysis .....                               | 11 |

|                                              |    |
|----------------------------------------------|----|
| Misclassification Summary.....               | 11 |
| Sample Misclassified Reviews.....            | 11 |
| Root Causes and Suggested Improvements ..... | 12 |
| Real-Time Prediction Interface.....          | 12 |
| Conclusion and Future Scope .....            | 13 |
| Summary of Findings.....                     | 13 |
| Limitations .....                            | 13 |
| Future Work.....                             | 13 |
| References.....                              | 13 |

## Table of Figures

|                                                                                                                                                   |    |
|---------------------------------------------------------------------------------------------------------------------------------------------------|----|
| Figure 1: The distribution of ratings (left) and type of sentiment class (right) on the 12,000 book reviews.<br>.....                             | 5  |
| Figure 2: Negative (red), Neutral (yellow) and Positive (green) sentiment words as word clouds. ....                                              | 5  |
| Figure 3: Histogram of the length of the review tokens displayed after preprocessing. Red line indicates<br>MAX LEN = 187 (95 th percentile)..... | 6  |
| Figure 4: Comparison of the bar charts of accuracy in test (left) and the time in seconds of training (right)<br>of all three models.....         | 7  |
| Figure 5: Validation loss (left) and validation accuracy (right) curves versus training epochs in all three<br>models.....                        | 8  |
| Figure 6: Simple RNN (left), BiLSTM Trainable (centre) and BiLSTM + Word2Vec (right) confusion<br>matrices.....                                   | 10 |
| Figure 7: Gradio-based GUI that provides the user with an opportunity to write a book review and obtain a<br>real-time sentiment prediction. .... | 12 |

## Abstract

This report describes an end-to-end pipeline of a sentiment analysis applied to 12,000 Amazon book reviews. The aim of the research was to categorize each of the reviews as Negative, Neutral, or Positive using three deep learning networks: A Simple Recurrent Neural Network (RNN) with a trainable embedding layer, a Bidirectional Long Short-Term Memory (BiLSTM) network with a trainable embedding layer, and a BiLSTM network initialized with custom Word2Vec embeddings trained on the review corpus. Preprocessing pipeline consisted of contraction expansion, URL removal, stopword filtering with sentiment preserving exceptions and WordNet lemmatization. Stratified sampling was used in splitting the dataset into train, validation and test sets of 70/15/15. The test accuracies of all three models were within the 71-72% range with the BiLSTM (Trainable Embedding) model recording the highest F1-score of Neutral-class of 0.38. It also implemented a real-time prediction interface implemented in Gradio.

Github Link: <https://github.com/samartha8/6th-Sem-Deep-Learning>

## Introduction

Natural Language Processing (NLP) is a sub-specialty of artificial intelligence related to empowering machines to comprehend, interpret, and generate human language (Muhammad, et al., 2021). Sentiment analysis - the automatic categorization of the expressed opinion in a piece of text as a positive, negative or a neutral opinion. Due to the rapid increase in the number of user content generated on websites like Amazon, online travel agencies, and social media, the size of the reviews and feedback is so large that it cannot be analyzed manually, and in this case, automated sentiment classification is an essential tool that both business and researchers can and ought to utilize (B G & Patil, 2024). Word embedding methods such as Word2Vec turn raw text into dense numeric vectors that are able to capture semantic relationships between words and Recurrent Neural Networks (RNNs) and Long Short-Term Memory (LSTM) networks are specifically created to model sequential dependencies in text data (Muhammad, et al., 2021).

This project uses the following NLP techniques to work with a dataset of 12,000 Amazon book reviews, the aim of which is to categorize each of the reviews into one of three sentiment categories: Negative, Neutral, or Positive. Three deep learning models are compared and implemented: a Simple RNN, a Bidirectional LSTM (BiLSTM) with a trainable embedding layer, and a BiLSTM initialized with custom Word2Vec embeddings - with a full text preprocessing pipeline, tokenization and a real time prediction interface built with Gradio.

## Dataset

### Source and Overview

The data set is the Book review.csv file that had 12,000 Amazon book reviews. The records are organized into three columns:

- Rating (an integer, 1 to 5)
- ReviewText (raw review text)
- Summary (short title)

None of the rows had been deleted in the column of reviewText, two rows had not had their summaries, however, this had no effect as only reviewText had been used.

| Attribute      | Value            | Notes                         |
|----------------|------------------|-------------------------------|
| Total Records  | 12,000           | After null drop: 12,000       |
| Feature Used   | reviewText       | Rating used for labels only   |
| Rating Range   | 1–5 (integer)    | Mapped to 3 sentiment classes |
| Missing Values | 2 (summary only) | No impact on model            |

## Sentiment Label Mapping

Star rating had been plotted on three sentiment classes. Ratings 1–2 → Negative (label 0), rating 3 → Neutral (label 1), ratings 4–5 → Positive (label 2). The problem that this project will focus on is a three-class problem with the ambiguous nature of the Neutral class, which is underrepresented.

| Rating | Sentiment | Label | Count       |
|--------|-----------|-------|-------------|
| 1–2    | Negative  | 0     | 4,000 (33%) |
| 3      | Neutral   | 1     | 2,000 (17%) |
| 4–5    | Positive  | 2     | 6,000 (50%) |

## Methodology

### Text Preprocessing

The preprocessing pipeline was as follows, using a `clean_text()` function:

- Lowercasing: Text was all lowercased to standardize the matching of the tokens later.
- Contraction expansion: Among the expansion forms in the contraction’s library, expansion forms that preserve the negation semantics, which is vital in the accuracy of sentiment, are listed.
- Removal of URL, mention and hashtag: Noise tokens (`http:` mention, hashtag) were removed via regular expression.
- Elimination of numbers and punctuations: The non-alphabetic characters were stripped.
- Stopword removal with exceptions: NLTK stopwords were filtered, but negation context-bearing words (not, no, never, nor, however, but, very, too, only, although, neither, without) were retained to ensure that negation context is not thrown away.
- Lemmatization: WordNet lemmatization minimized tokens to base forms (e.g., books → book). Instead of stemming, lemmatization was chosen as it results in valid vocabulary words, enhancing the quality of embeddings in Word2Vec.

## Sequence Length Analysis

Mean length of reviews (in cleaned tokens) was 56 words with a standard deviation of 65, with the highest number of words being 1,129. The 95 th percentile was 187 tokens, and this was considered as MAX\_LEN. The computationally wasteful way would have been to pad all sequences to the maximum; the 95th-percentile would have retained the vast majority of token information whilst keeping the sequence matrix manageable.

## Tokenization and Padding

The Keras Tokenizer was trained only on training data to avoid the leakage of the information during the validation or test sets training. A vocabulary of VOCABsize = 1 2,000 most frequent tokens was retained, and an OOV token is chosen to index 1. The validation and test OOV rates were 6.5 and 6.7 respectively. After sequence, the sequences were padded to MAX\_LEN = 187.

## Splits and Class Weights

The data was broken into stratified splits to have similar classes proportion. The class weights were calculated with the loved balanced mode of the sklearn library to prevent the underrepresented Neutral class (17%): Negative = 1.0, Neutral = 2.0, Positive = 0.667. These weights were then handed over to model.fit to be sure each class would contribute proportional contributions to the loss.

| Split      | Size  | Proportion | Purpose                                |
|------------|-------|------------|----------------------------------------|
| Training   | 8,400 | 70%        | Model fitting                          |
| Validation | 1,800 | 15%        | Early stopping / hyperparameter tuning |
| Test       | 1,800 | 15%        | Final held-out evaluation              |

## Model Architectures

### Model 1 – Simple RNN with Trainable Embedding

A baseline was used with the first model. An Embedding layer (VOCAB\_size=12,000, dim=64) with L2 regularisation was followed by SpatialDropout1D (0.37), a SimpleRNN layer (48 units, dropout=0.27, recurrent\_dropout=0.13, L2 regularisation), two Dense layers with Dropout(0.55) and Dropout(0.45) and a

3-class softmax output. The vanishing gradient problem was alleviated using Adam (lr=0.0003, clipnorm=1.0).

## Model 2 – BiLSTM with Trainable Embedding

In the second model, the SimpleRNN was replaced with a Bidirectional LSTM (BiLSTM). The gated cell memory of LSTM, which includes an input gate, forget gate, and output gate, enables it to selectively retain or forget information across long sequences overcoming the vanishing gradient problem. The Bidirectional wrapper reads sequences both forwards and backwards giving more contextual representations. To overcome overfitting, increased regularization (SpatialDropout1D=0.40, LSTM dropout=0.50, recurrent\_dropout=0.30, L2=5e-4) was introduced.

## Model 3 – BiLSTM with Word2Vec Embeddings

The third model was a variation of Model 2 that uses a custom Word2Vec embedding layer that is trained via the training split using Gensim. This model had Skip-gram(sg=1), window=5, min count=2 and 15 training epochs with vector size=100. It is almost perfect (at a student level) with 100 percent Word2Vec coverage (11,998 out of 11,999 vocabulary words). The embedding layer was retained trainable to be fine-tuned at the time of backpropagation.

| Component       | Model 1 (SimpleRNN) | Model 2 (BiLSTM)  | Model 3 (BiLSTM+W2V) |
|-----------------|---------------------|-------------------|----------------------|
| Embedding       | Trainable, dim=64   | Trainable, dim=64 | Word2Vec, dim=100    |
| Spatial Dropout | 0.37                | 0.40              | 0.20                 |
| Recurrent Layer | SimpleRNN(48)       | BiLSTM(32)        | BiLSTM(32)           |
| LSTM Dropout    | 0.27 / 0.13         | 0.50 / 0.30       | 0.30 / 0.15          |
| Dense           | 16 units, ReLU      | 16 units, ReLU    | 16 units, ReLU       |
| Output          | Softmax (3-class)   | Softmax (3-class) | Softmax (3-class)    |
| Optimiser LR    | Adam 0.0005         | Adam 0.0001       | Adam 0.0003          |
| Loss Function   | Sparse Cat. CE      | Sparse Cat. CE    | Sparse Cat. CE       |





## Review Length Distribution

Figure 3 provides the distribution of the length of clean reviews. MAX\_LEN = 187 is the red dashed line and is selected as the 95th percentile. The long right tail (to 1,129 tokens) supports the idea that padding to maximum length would have been a waste of resources.

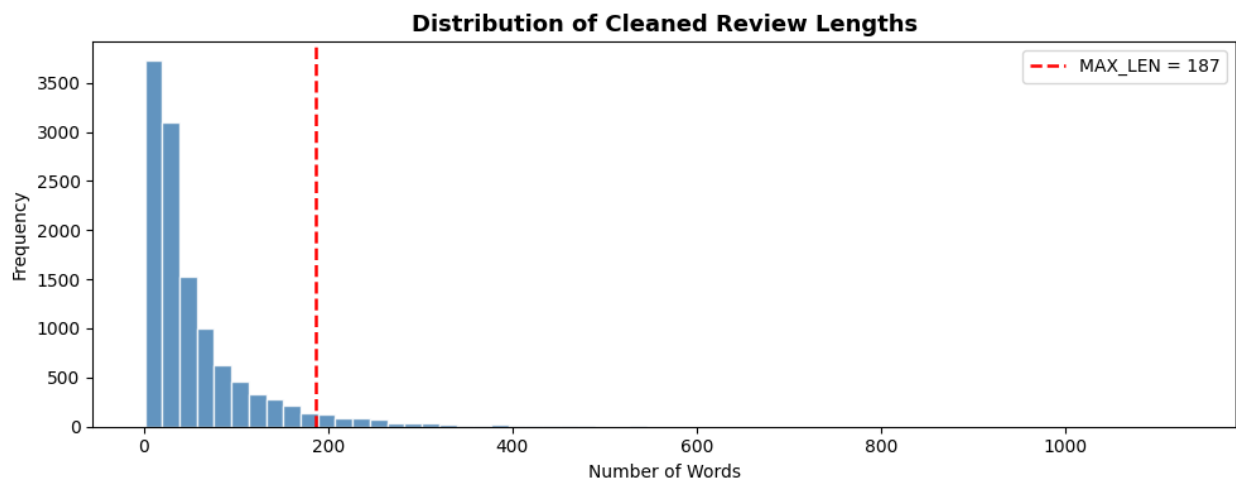


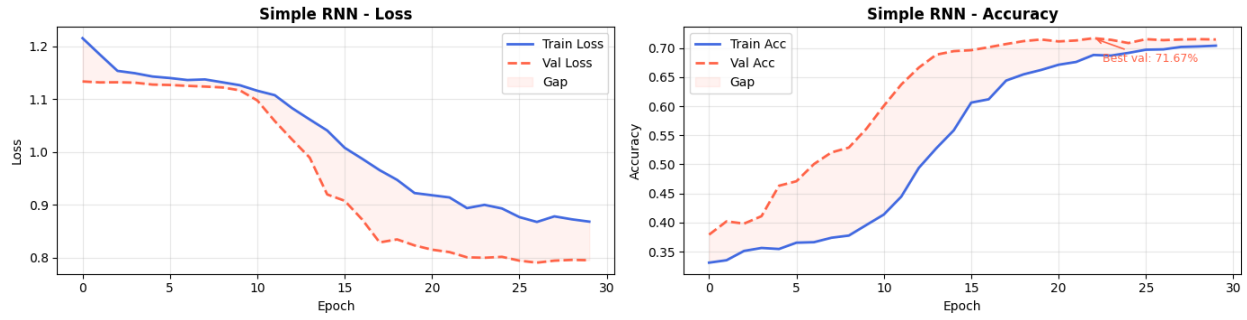
Figure 3: Histogram of the length of the review tokens displayed after preprocessing. Red line indicates MAX\_LEN = 187 (95th percentile).

## Overall Model Performance

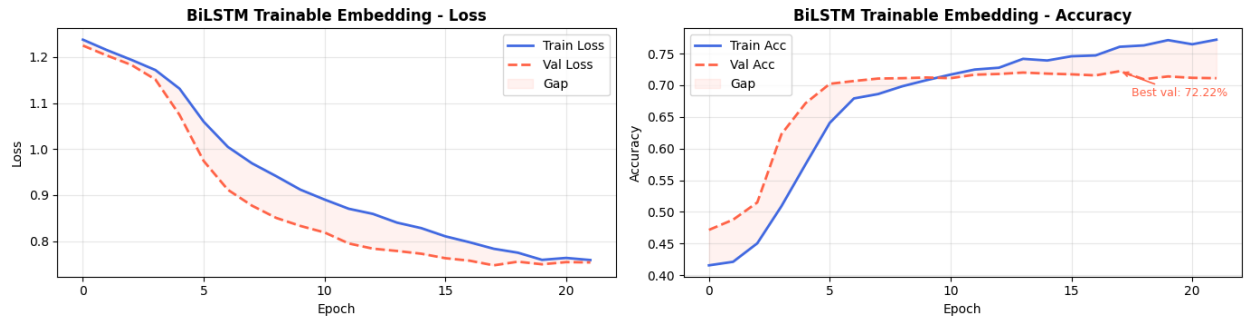
EarlyStopping (restore best weights = True) and class weights were used to train all three models. The overall performance of the final test set is summarized in Table 4. The range of 71 -72 indicates the natural challenge of three-classification with an unclear Neutral category.

| Model                        | Test Acc. | Train Time | Epochs | Best Val Acc. |
|------------------------------|-----------|------------|--------|---------------|
| Model 1 – Simple RNN         | 71.33%    | 525.8 s    | ~14    | ~72%          |
| Model 2 – BiLSTM (Trainable) | 72.06%    | 1668.0 s   | ~10    | ~72%          |
| Model 3 – BiLSTM + Word2Vec  | 71.67%    | 2066.1 s   | ~15+   | ~73%          |

Simple RNN | Best Val Acc: 71.67% | Train/Val gap: 1.06% (Healthy)



BiLSTM Trainable Embedding | Best Val Acc: 72.22% | Train/Val gap: 6.08% (Mild overfit)



BiLSTM + Word2Vec | Best Val Acc: 71.56% | Train/Val gap: 1.52% (Healthy)

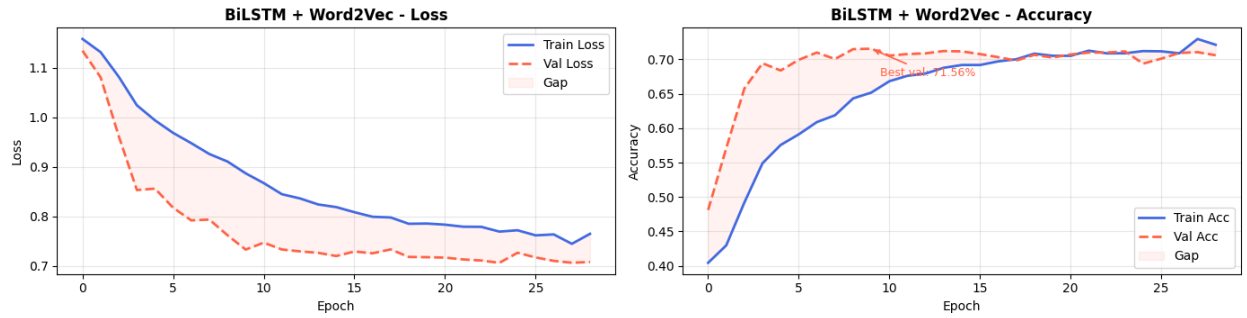


Figure 4: Comparison of the bar charts of accuracy in test (left) and the time in seconds of training (right) of all three models.

## Training Curves

Overlaid in Figure 5, are training loss, validation loss and validation accuracy curves of all three models. The lengths of the x-axis vary since EarlyStopping terminated each of the models at a different epoch. Simple RNN converged the quickest, and the BiLSTM + Word2Vec model enjoyed the advantage of many more training epochs before leveling off due to the enhanced richness of initialization of the behavioral rule via the rich initialization of Word2Vec embeddings.

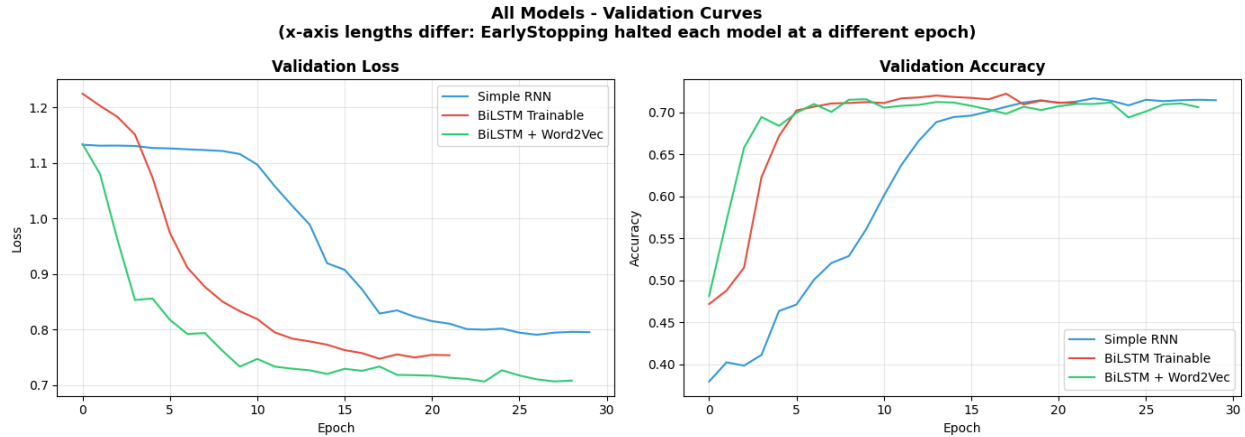


Figure 5: Validation loss (left) and validation accuracy (right) curves versus training epochs in all three models.

## Per-Class Performance

### Model 1 – Simple RNN

| Class                   | Precision | Recall | F1-Score | Support |
|-------------------------|-----------|--------|----------|---------|
| Negative                | 0.69      | 0.84   | 0.75     | 600     |
| Neutral                 | 0.22      | 0.05   | 0.09     | 300     |
| Positive                | 0.77      | 0.85   | 0.81     | 900     |
| <b>Overall Accuracy</b> | 71.33%    |        |          | 1800    |

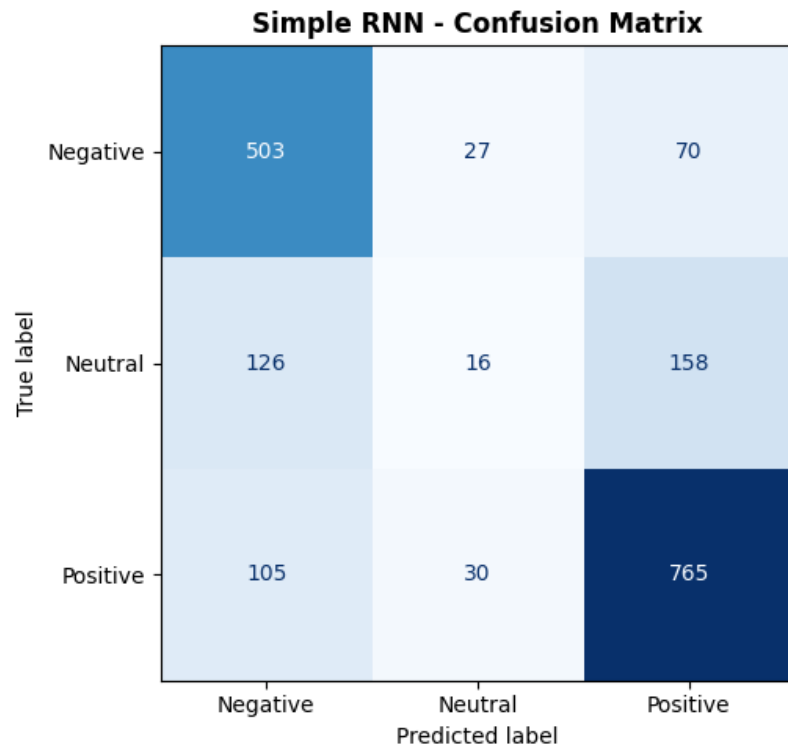
### Model 2 – BiLSTM Trainable Embedding

| Class                   | Precision | Recall | F1-Score | Support |
|-------------------------|-----------|--------|----------|---------|
| Negative                | 0.74      | 0.79   | 0.76     | 600     |
| Neutral                 | 0.28      | 0.22   | 0.24     | 300     |
| Positive                | 0.82      | 0.84   | 0.83     | 900     |
| <b>Overall Accuracy</b> | 72.06%    |        |          | 1800    |

### Model 3 – BiLSTM + Word2Vec

| Class                   | Precision | Recall | F1-Score | Support |
|-------------------------|-----------|--------|----------|---------|
| Negative                | 0.80      | 0.76   | 0.78     | 600     |
| Neutral                 | 0.33      | 0.45   | 0.38     | 300     |
| Positive                | 0.84      | 0.78   | 0.81     | 900     |
| <b>Overall Accuracy</b> | 71.67%    |        |          | 1800    |

The confusion matrices of all the three models have been presented in total as shown in figure 6. Model 3 had the best-balanced Neutral recall of 45% as opposed to only 5% with Model 1 and 22 percent with Model 2.



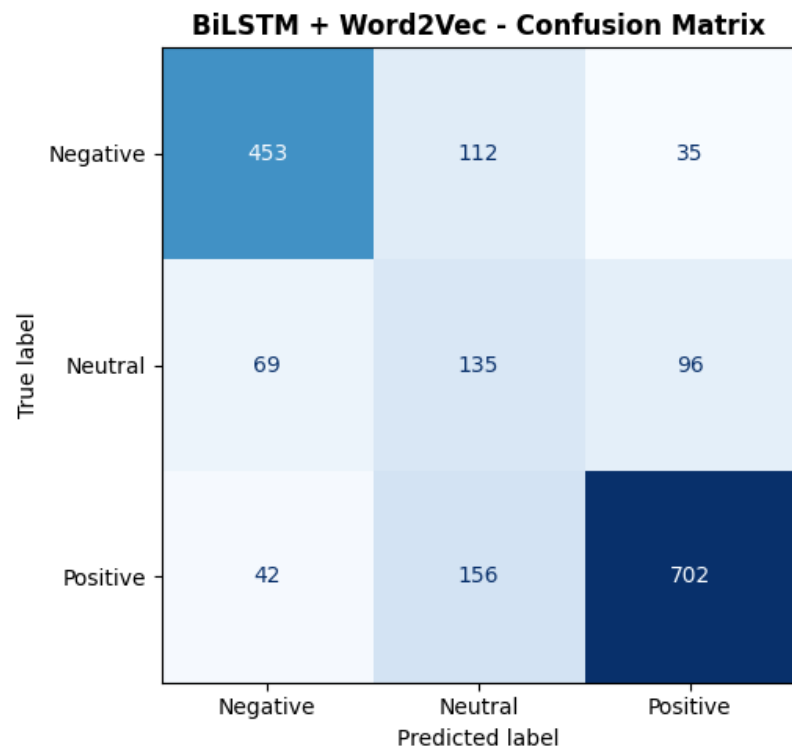
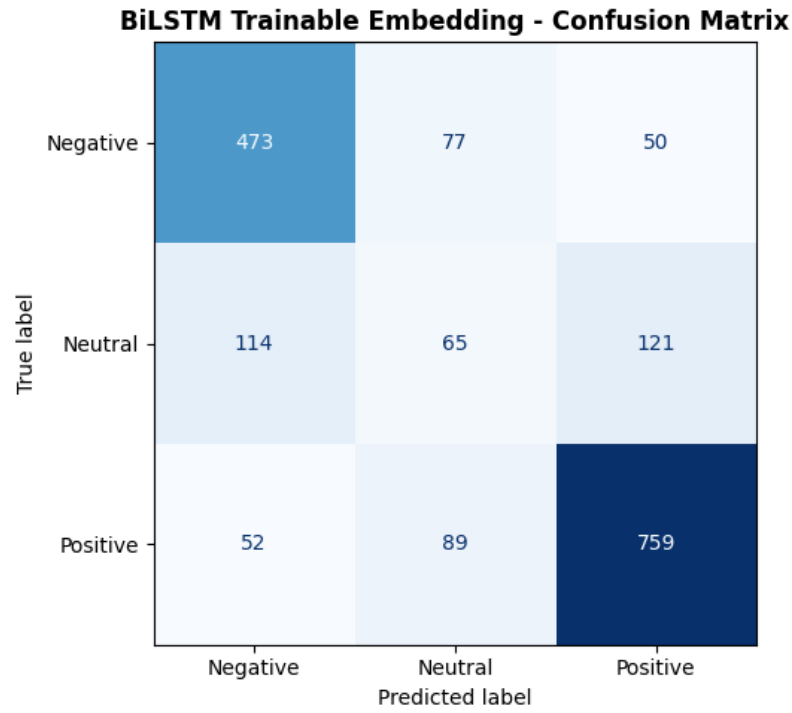


Figure 6: Simple RNN (left), BiLSTM Trainable (centre) and BiLSTM + Word2Vec (right) confusion matrices.

## Embedding Comparison

In model 1 and model 2, randomly initialised trainable trainee embeddings (dim=64) were used. Model 3 tested word2vec vectors trained on the review corpus (dim=100) with near-perfect coverage of the vocabulary with 100% coverage (11,998 of 11,999 words). The Word2Vec benefit was most evident in the Neutral recall showing that the richer the initial word representations, the less likely it is to confuse mixed-sentiment language.

## Computational Efficiency

Simple RNN took an approximate of 526 seconds to complete training, compared to 1,668 seconds in the case of BiLSTM (Trainable) and 2,066 seconds in the case of BiLSTM + Word2Vec. All the training was performed on Google Colab with gpu acceleration (NVIDIA T4).

## Error Analysis

### Misclassification Summary

Model 3 (BiLSTM + Word2Vec) was analyzed in terms of errors. Among the 1,800 test samples, 510 were misclassified (28.3% error percentage). The breakdown per-class is illustrated hereunder:

| Class    | Total Samples | Errors | Error Rate |
|----------|---------------|--------|------------|
| Negative | 600           | 147    | 24.5%      |
| Neutral  | 300           | 165    | 55.0%      |
| Positive | 900           | 198    | 22.0%      |

### Sample Misclassified Reviews

Example 1: True: Neutral | Predicted: Positive. An overview of a so-called enjoyed plot in which the characters are developingly weak. A positive coverage of the embedding signal (positive words like enjoyed, good, etc.) overwhelmed the negative critical commentaries.

Examples 2 and 3 had similar trends: mixed-vocabour reviews (praise plus criticism) were pushed towards the majority Positive class, and short reviews (less than 5 cleaned tokens) offered a poor context in which to meaningfully model such sequences.

## Root Causes and Suggested Improvements

Truncation on neutral vocabulary: Reviews in the 3-star range both use the language of both the positive and negative categories, and creates a fuzzy boundary in decision making despite the lack of attention mechanisms. Inadequacy of short reviews: Reviews with less than 10 clean tokens do not provide adequate context to be able to model sequences. This could be met by a minimum length filter (or special short-text branch). Sarcasm and irony: Phrases that have positive surface form yet negative intention cannot be detected by using static embeddings of words at the surface.

Potential enhancements include focal loss to upweight hard-to-classify Neutral samples, oversampling with a synonym-based augmentation, or its replacement by a transformer-based model such as BERT or RoBERTa.

## Real-Time Prediction Interface

The GUI of a prediction in real-time was provided by the Gradio library. The most successful model (BiLSTM + Word2Vec) was saved as model in Keras and the Keras Tokenizer serialised using the pickle library in Python. The two artefacts were re-read in a self-contained Gradio application. The `clean_text()` preprocessing method was copied in such a way that, inference-time input is cleansed just like training data. To create a public URL that can be used to demonstrate Gradio, the interface was started with `share=True`.

**Book Review Sentiment Classifier**

BiLSTM + Word2Vec sentiment classifier for book reviews.

review\_text

One of the best books I have ever read. The author has a gift for storytelling and every chapter left me wanting more. A masterpiece of modern fiction.

Clear Submit

Predicted sentiment

Positive

Confidence

0.400953084230423

Class probabilities

**Positive**

|          |     |
|----------|-----|
| Positive | 46% |
| Negative | 32% |
| Neutral  | 28% |

Figure 7: Gradio-based GUI that provides the user with an opportunity to write a book review and obtain a real-time sentiment prediction.



## Conclusion and Future Scope

### Summary of Findings

The project was able to implement three deep learning models and compare the models against each other in three-class sentiment classification of Amazon book reviews. The test accuracy of all the models was in the 71.72 %. The highest overall model was Model 2 (BiLSTM, Trainable Embedding) which had the best overall accuracy of 72.06 % with the best Neutral-class recall values (45%) and the best F1-score (0.38). The Simple RNN was found to be the most computationally effective and as such, training on Simple RNN was about 3-4 times faster than BiLSTM variants. The Corpus of reviews (NIVE) trained Word2Vec embeddings demonstrated an improvement in Neutral-class detection.

### Limitations

- The imbalance structure provided by the three-class formulation cannot be effectively eliminated by the use of the weights of the classes.
- Domain-specific Word 2 Vec was applied as opposed to larger scale external embeddings, larger, pre-trained embeddings could still enhance results.
- None of the models can effectively deal with sarcasm or context-dependent sentiment.

### Future Work

- Use focal loss, or neutral-class F1 to enhance Neutral-class F1.
- Test transformer-based systems including BERT/RoBERTa to gain more contextual representation.  
Break: Observe the following consequences on all three classes of sentiment when you increase Word2Vec dimension to 300 and evaluate the effect on all three classes of sentence.
- Increase the size of the dataset by adding more Neutral reviews as a way of reducing the class imbalance.

## References

- B G, P. & Patil, V., 2024. *Enhanced Sentiment Analysis of Airline Twitter Review Using Hybrid Machine Learning and Deep Learning Models*. s.l., IEEE.
- Muhammad, P. F., Kusumaningrum, R. & Wibowo, A., 2021. Sentiment Analysis Using Word2vec And Long Short-Term Memory (LSTM) For Indonesian Hotel Reviews. *Procedia Computer Science*, Volume 179, pp. 728-735.